

# Communicators

Victor Eijkhout  
Jack Gaither

2026 PCSE

# Overview

In this section you will learn about various subcommunicators.

Commands learned:

- *MPI\_Comm\_dup*, discussion of library design
- *MPI\_Comm\_split*
- discussion of groups
- discussion of inter/intra communicators.

# Sub-computations

Simultaneous groups of processes, doing different tasks, but loosely interacting:

- Simulation pipeline: produce input data, run simulation, post-process.
- Climate model: separate groups for air, ocean, land, ice.
- Quicksort: split data in two, run quicksort independently on the halves.
- Process grid: do broadcast in each column.

New communicators are formed recursively from `MPI_COMM_WORLD`.

# Communicator duplication

Simplest new communicator: identical to a previous one.

```
1 int MPI_Comm_dup(MPI_Comm comm, MPI_Comm *newcomm)
```

This is useful for library writers:

```
1 MPI_Isend(...); MPI_Irecv(...);  
2 // library call  
3 MPI_Waitall(...);
```

- Naively, the library can 'catch' the user messages.
- With a duplicate communicator there is no confusion: user and library both have their own 'context' for their messages.

# Interleaved library and user code

```
1 library my_library(comm);  
2 MPI_Isend(&sdata,1,MPI_INT,other,1,comm,&(request[0]));  
3 my_library.communication_start();  
4 MPI_Irecv(&rdata,1,MPI_INT,other,MPI_ANY_TAG,  
5          comm,&(request[1]));  
6 MPI_Waitall(2,request,status);  
7 my_library.communication_end();
```

# Library internally has messages

```
1 int library::communication_start() {
2     int sdata=6,rdata;
3     MPI_Isend(&sdata,1,MPI_INT,other,2,comm,&(request[0]));
4     MPI_Irecv(&rdata,1,MPI_INT,other,MPI_ANY_TAG,
5              comm,&(request[1]));
6     return 0;
7 }
8
9 int library::communication_end() {
10    MPI_Status status[2];
11    MPI_Waitall(2,request,status);
12    return 0;
13 }
```

# Wrong way of setting up the library

```
1 // commdupwrong.cxx
2 class library {
3 private:
4     MPI_Comm comm;
5     int procno,nprocs,other;
6     MPI_Request request[2];
7 public:
8     library(MPI_Comm incomm) {
9         comm = incomm;
10        MPI_Comm_rank(comm,&procno);
11        other = 1-procno;
12    };
13    int communication_start();
14    int communication_end();
15 };
```

# Right way of setting up the library

```
1 // commdupright.cxx
2 class library {
3 private:
4     MPI_Comm comm;
5     int procno,nprocs,other;
6     MPI_Request request[2];
7 public:
8     library(MPI_Comm incomm) {
9         MPI_Comm_dup(incomm,&comm);
10        MPI_Comm_rank(comm,&procno);
11        other = 1-procno;
12    };
13    ~library() {
14        MPI_Comm_free(&comm);
15    }
16    int communication_start();
17    int communication_end();
18 };
```



# Disjoint splitting

Split a communicator in multiple disjoint others.

Give each process a 'color', group processes by color:

```
1 int MPI_Comm_split(MPI_Comm comm, int color, int key,  
2 MPI_Comm *newcomm)
```

(key determines ordering: use rank unless you want special effects)

# MPI\_Comm\_split

| Name             | Param name | Explanation                  | C type    | F type         |
|------------------|------------|------------------------------|-----------|----------------|
| MPI_Comm_split ( |            |                              |           |                |
|                  | comm       | communicator                 | MPI_Comm  | TYPE(MPI_Comm) |
|                  | color      | control of subset assignment | int       | INTEGER        |
|                  | key        | control of rank assignment   | int       | INTEGER        |
|                  | newcomm    | new communicator             | MPI_Comm* | TYPE(MPI_Comm) |
|                  | )          |                              |           |                |

# MPL: MPI\_Comm\_split

```
1 template<typename color_type, typename key_type = int>
2 mpl::communicator(
3     mpl::communicator::split_tag split,
4     const mpl::communicator &other,
5     color_type color,
6     key_type key = 0);
```

# Row/column example

Simulate a process grid

create subcommunicator per column (or row)

assume processes numbered by rows

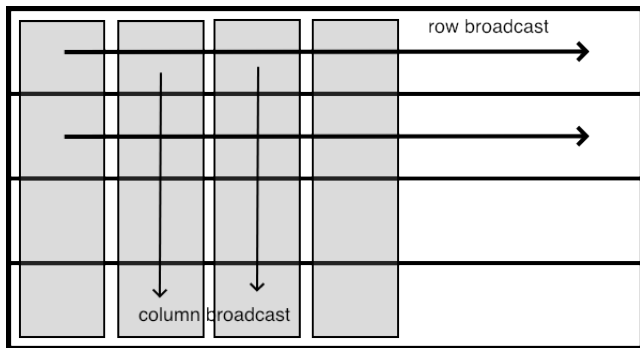
```
1 MPI_Comm_rank( MPI_COMM_WORLD, &procno );  
2 proc_i = procno % proc_column_length;  
3 proc_j = procno / proc_column_length;  
4  
5 MPI_Comm column_comm;  
6 MPI_Comm_split( MPI_COMM_WORLD, proc_j, procno, &column_comm );  
7  
8 MPI_Bcast( data, ... column_comm );
```

Food for thought: there are many columns, but only one `column_comm` variable. Why?

# MPL row/col

```
1 mpl::communicator col_comm  
2   (mpl::communicator::split_tag(),comm_world,  
3     proc_j,procno // that last one is optional  
4   );
```

# Row and column communicators



Row and column broadcasts in subcommunicators

# Exercise 1 procgrid

Organize your processes in a grid, and make subcommunicators for the rows and columns. For this compute the row and column number of each process.

In the row and column communicator, compute the rank. For instance, on a  $2 \times 3$  processor grid you should find:

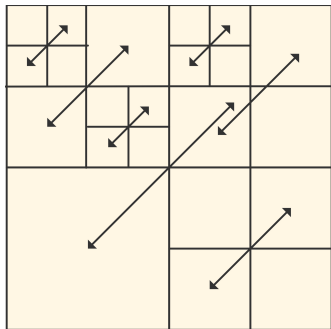
| Global ranks: | Ranks in row: | Ranks in column: |
|---------------|---------------|------------------|
| 0 1 2         | 0 1 2         | 0 0 0            |
| 3 4 5         | 0 1 2         | 1 1 1            |

Check that the rank in the row communicator is the column number, and the other way around.

Run your code on different number of processes, for instance a number of rows and columns that is a power of 2, or that is a prime number. This is one occasion where you could use `ibrun -np 9`; normally you would *never* put a processor count on `ibrun`.

## Exercise 2

Implement a recursive algorithm for matrix transposition:



- Swap blocks (1, 2) and (2, 1); then
- Divide the processors into four subcommunicators, and apply this algorithm recursively on each;
- If the communicator has only one process, transpose the matrix in place.



# Splitting by shared memory

- *MPI\_Comm\_split\_type* splits into communicators of same type.
- MPI-3: only *MPI\_COMM\_TYPE\_SHARED* splitting by shared memory.
- MPI-4: *MPI\_COMM\_TYPE\_HW\_GUIDED* split using an *info* value from *MPI\_Get\_hw\_resource\_types*.

```
1 // commsplitttype.c
2 MPI_Info info;
3 MPI_Comm_split_type
4   (MPI_COMM_WORLD,
5    MPI_COMM_TYPE_SHARED,
6    procno, info, &sharedcomm);
7 MPI_Comm_size
8   (sharedcomm, &new_nprocs);
9 MPI_Comm_rank
10  (sharedcomm, &new_procno);
```

# More

- Non-disjoint subcommunicators through process groups.
- Process topologies: cartesian and graph.  
There will also be a section about this, later.

## Cartesian topologies

# Cartesian decomposition

Code:

```
1 // cartdims.c
2 int *dimensions = (int*) malloc(dim*
    sizeof(int));
3 for (int idim=0; idim<dim; idim++)
4     dimensions[idim] = 0;
5 MPI_Dims_create(nprocs,dim,dimensions);
```

Output:

```
1 mpicc -o cartdims
    ↪ cartdims.o
2 Cartesian grid size: 3
    ↪ dim: 1
3 3
4 Cartesian grid size: 3
    ↪ dim: 2
5 3 x 1
6 Cartesian grid size: 4
    ↪ dim: 1
7 4
8 Cartesian grid size: 4
    ↪ dim: 2
9 2 x 2
10 Cartesian grid size: 4
    ↪ dim: 3
11 2 x 2 x 1
12 Cartesian grid size: 12
    ↪ dim: 1
13 12
14 Cartesian grid size: 12
    ↪ dim: 2
15 4 x 3
16 Cartesian grid size: 12
```

# Create/test Cartesian topology

```
1 MPI_Comm cart_comm;
2 int *periods = (int*) malloc(dim*sizeof(int));
3 for ( int id=0; id<dim; id++ ) periods[id] = 0;
4 MPI_Cart_create
5   ( comm,dim,dimensions,periods,
6     0,&cart_comm );

1 int dim;
2 MPI_Cartdim_get( cart_comm,&dim );
3 int *dimensions = (int*) malloc(dim*sizeof(int));
4 int *periods    = (int*) malloc(dim*sizeof(int));
5 int *coords     = (int*) malloc(dim*sizeof(int));
6 MPI_Cart_get( cart_comm,dim,dimensions,periods,coords );
```

# MPL dims create

```
1 mpl::cartesian_communicator::dimensions brick(3);  
2 brick = mpl::dims_create(nprocs,brick);  
1 mpl::cartesian_communicator cart_comm( comm_world,brick );
```

# Rank translation

```
1 // cart.c
2 MPI_Comm comm2d;
3 int periodic[ndim]; periodic[0] = periodic[1] = 0;
4 MPI_Cart_create(comm,ndim,dimensions,periodic,1,&comm2d);
5 if (comm2d==MPI_COMM_NULL) {
6     printf("Process %d not included\n",procno);
7 } else {
8     MPI_Cart_coords(comm2d,procno,ndim,coord_2d);
9     MPI_Cart_rank(comm2d,coord_2d,&rank_2d);
10    printf("I am %d: (%d,%d); originally %d\n",
11           rank_2d,coord_2d[0],coord_2d[1],procno);
```

# Cartesian communication

```
1 // cartcoord.c
2 for ( int id=0; id<dim; id++)
3   periods[id] = id==0 ? 1 : 0;
4 MPI_Cart_create
5   ( comm,dim,dimensions,periods,
6     0,&period_comm );
```

Code:

```
1 int pred,succ;
2 MPI_Cart_shift
3   (period_comm,/* dim: */ 0,/* up: */ 1,
4     &pred,&succ);
5 printf
6   ("periodic dimension 0:\n  src=%d, tgt
   =%d\n",
7     pred,succ);
8 MPI_Cart_shift
9   (period_comm,/* dim: */ 1,/* up: */ 1,
10    &pred,&succ);
11 printf
12   ("non-periodic dimension 1:\n  src=%d,
   tgt=%d\n",
13    pred,succ);
```

Output:

```
1 Grid of size 6 in 3
   ↪ dimensions:
2   3 x 2 x 1
3 Shifting process 0.
4 periodic dimension 0:
5   src=4, tgt=2
6 non-periodic dimension
   ↪ 1:
7   src=-1, tgt=1
```



# MPL Cartesian shifting

```
1 int pred,succ;  
2 mpl::shift_ranks shifted = cart_comm.shift  
3   ( /* dim: */ 1,/* up: */ 1 );  
4 pred = shifted.source; succ = shifted.destination;
```

# Subgrids

Code:

```
1 MPI_Cart_sub( period_comm,remain,&
    hyperplane );
2 if (procno==0) {
3     MPI_Topo_test( hyperplane,&topo_type );
4     MPI_Cartdim_get( hyperplane,&hyperdim )
    ;
5     printf("hyperplane has dimension %d,
    type %d\n",
6         hyperdim,topo_type);
7     MPI_Cart_get( hyperplane,dim,dims,
    period,coords );
8     printf(" periodic: ");
9     for (int id=0; id<2; id++)
10         printf("%d,",period[id]);
11     printf("\n");
```

Output:

```
1 Grid of size 6 in 3
    ↪dimensions:
2     3 x 2 x 1
3 hyperplane has
    ↪dimension 2,
    ↪type 2
4 periodic: 1,0,
```

## Exercise 3 isendirecvcart

Use Cartesian topology routines to extend exercise ?? to two dimensions.

## Inter-communicators

# Inter-communicators

- Communicators so far are of *intra-communicator* type.
- Bridge between two communicators: *inter-communicator*.
- Example: communicator with newly spawned processes

# In a picture



## Illustration of ranks in an inter-communicator setup

```
1 // intercomm.c
2 MPI_Comm intercomm;
3 MPI_Intercomm_create
4   (/* local_comm:      */ split_half_comm,
5    /* local_leader:    */ local_leader_in_inter_comm,
6    /* peer_comm:       */ MPI_COMM_WORLD,
7    /* remote_peer_rank: */ global_rank_of_other_leader,
8    /* tag:             */ inter_tag,
9    /* newintercomm:     */ &intercomm );
```

# Concepts

- Two local communicators
- The 'peer' communicator that contains them
- Leaders in each of them
- An inter-communicator over the leaders.

# Routines

- *MPI\_Intercomm\_create*: create
- *MPI\_Comm\_get\_parent*: the other leader (see process management)
- *MPI\_Comm\_remote\_size*, *MPI\_Comm\_remote\_group*: query the other communicator
- *MPI\_Comm\_test\_inter*: is this an inter or intra?